NATIONAL INSTISTUTE OF BUSINESS MANAGEMENT

SCHOOL OF COMPUTING & ENGINEERING



ROBOTICS COURSEWORK

AUTOMATED WATER TANK

HIGHER DIPLOMA IN SOFTWARE ENGINEERING

26.1F

COHNDSE26.1F – 004

K.G.L. Sandanimne

Contents

Abstract 1

Acknowledge 2

Table of Contents .....

Table of Figures .....

Chapter 01- INTRODUCTION

Project Introduction

- **1.1 Project Background** 3
- **1.2 Problem Statement** **3**
- **1.3 Aim and Objectives** **4**

Literature Review & Requirements Engineering

- **2.1 Review of Automated Fluid Management Systems** **5**
- **2.2 Comparative Study of Level-Sensing Technologies** **5**
- **2.3 Requirements Analysis** **6**

Chapter 02- METHODOLOGY

Hardware Architecture & Electrical Design

- **3.1 System Hardware Overview** **7**
- **3.2 Component Selection & Justification** **7**
- **3.3 Power Distribution Grid & Electrical Safety** **14**
- **3.4 Hardware Pinout Matrix & Schematic Interconnection** **15**

UI/UX & SCADA Supervisory Control

- **4.1 High-Density Heads-Up Display (HUD) Design** **15**
- **4.2 SCADA Serial Command Interface** **16**

Discussion..... **18**

Future Implementation **19**

Reference **22**

Time Frame **20**

Appendix..... **23**

Table of Figures

Figure 3.1: 30-pin ESP32 Microcontroller 07

Figure 3.2: ESP32 Expansion Board 08

Figure 3.3: AJ-SR04T Waterproof Ultrasonic Sensor 09

Figure 3.4: YF-S201 Water Flow Sensor 10

Figure 3.5: 12V Solenoid Valve and Submersible DC Pump Actuator 10

Figure 3.6: 4 Channel 12V Relay Module 11

Figure 3.7: 0.96 Inches SSD1306 I2C OLED Display 11

Figure 3.8: Active Buzzer Module and Normal Package of LED 12

Figure 3.9: LM2596 Step-Down Voltage Buck Regulator 12

Figure 3.10: Female DC Barrel Power Supply Jack 13

Figure 3.11: Jumper wires / AWG 18 wires 13

Figure 5.1: 0.96-inch Industrial OLED HUD Layout and Zones 15

Figure 8.1: 4-Week Project Timeline (Gantt Chart) 20

Figure 9.2: Cirkit Designer Wiring Schematics ……..... 23

ABSTRACT

An innovative solution for automating fluid control systems in industry and in households has identified some of the major limitations of the conventional mechanical switch controls, namely, mechanical fragility, lack of telemetry and ability to identify hardware risks that can damage the pump and result in fluid loss. In this paper, we develop an innovative fluid control system using an event-driven system controlled by a 32-bit ESP32 MCU.

Besides hardware considerations, the embedded software provides an extremely reliable SCADA system.In the described prototype, ultrasonic measurement of water level and Hall effect measurement of fluid flow are managed by an explicit eight-state finite-state machine with non-blocking behavior. In order to increase reliability in an industrial environment, the software utilizes some signal processing methods such as the implementation of a five-point moving average to reduce acoustic jitter and atomic variables to avoid the problem of race conditions due to frequent interrupts.

Software safety measures are introduced to increase system reliability, including the 60-second dry run of the pump protection system.Statuses and telemetry information are presented through an OLED interface that is based on a serial interface. The Manual Override functionality is available too. The developed system shows its effectiveness in overcoming the limitations of the traditional mechanical control automation systems.

Acknowledge

Our heartfelt thanks go to the Robotics Module Supervisor, **Mr. Supun Asanga**, for the valuable guidance and assistance given through the course of the project development. Our thanks are greatly due to him for his technical knowledge and advice in order to make this project happen successfully.In this context, we thank the members of the School of Computing and Engineering and the National Institute of Business Management for providing us with all the necessary resources for carrying out this project.

Further, we wish to thank our teammates in our project group for all the help that they have given us during the entire process of project development. They have done a great deal to make this project possible.

Chapter 1 - INTRODUCTION

1.1 Project Background

The proper management of water and other fluid resources represents one of the basic requirements for the infrastructure of the house and industry. Traditionally, the automated control of the fluid resources has relied on simple mechanical float switches. Though the use of traditional solutions is rather affordable and easy to integrate, it does not incorporate any intelligent mechanisms that would help with hardware diagnostics or adjustment to environmental changes.

In view of the development of Industry 4.0 and automated systems, there arose the need for precise and fault tolerant mechatronics. Modern software engineering offers the possibility to substitute mechanical switches with a deterministic microcontroller. The Automated Water Tank project covers this issue. Software state machines to adopt the principles of industrial SCADA system for local management of fluid resources.

1.2 Problem Statement

The ongoing use of old-fashioned mechanical automation technology for fluid handling system presents several major weaknesses that should be addressed by this project.

1.2.1 Vulnerabilities of Mechanical Float Switches

The mechanical float switch uses solely the effect of physical buoyancy with moving parts inside. These parts become susceptible to calcification, deposition of minerals, and mechanical jamming with time. The float switch being stuck in “ON” position can cause an overflow of the tank, while being stuck is “Off” position can lead to complete drainage of water from the system.

1.2.2 Unmonitored Pump Burnout & Dry-Run Risks

The submersible and inline pumps for water use require water itself for their cooling and lubrication. If, in the conventional setup, the request for the pump to start from the tank occurs while there is no water available in the supply line, the mechanical switch keep the pump running. The pump operation without the cooling water (“dry run”) leads to overheating, melting of seals and burning out. No sensing capabilities are inherent to the traditional setup.

1.2.3 Lack of Real-Time Telemetry and Diagnostic Control

Automated tanks follow the “black box” principle. No capability exists for real-time monitoring of processes such as flow rate and tank volume. Preventive maintenance is made difficult in the absence of real-time telemetry and manual manipulation of actuators.

1.3 Aim and Objectives

The fundamental aim of the project is to design an autonomous event driven system for controlling fluids that uses a software state machine, instead of the mechanical devices that can easily fail in the process of operation. In order to meet the above-mentioned aim, the following objectives have been set:

- To design a non-blocking FSM which can be able to poll for sensor data without becoming frozen.
- To develop mathematical filters for processing sensor signals to enable generation of reliable depth data from the raw sensor signals.
- To build industrial safety interlocks such as the one used for detecting dry run pumps and the gravity drain lock.
- To design an OLED graphical user interface (GUI) along with SCADA serial command interface.

Literature Review & Requirements Engineering

2.1 Review of Automated Fluid Management Systems

Water Management Automation Is an All-Pervasive Need in Residential as Well as Industrial Scenarios. Conventional water regulation usually uses simplistic physical means. The existing approach involves using a simple float valve that operates based on the principle of buoyancy of the water. While these systems are cheap and easy to implement, they do not have any "smart" feature: They do not give any feedback to the user and cannot monitor whether there is a breakage in the piping, or the supply tank is empty.

However, the modern-day industrial set-ups use Programmable Logic Controllers (PLC) in order to ensure precise fluid regulation. However, PLC is too costly and complicated to use in small scenarios. This project tries to find a solution to this problem by using the ESP32 microcontroller. ESP32 provides adequate processing power for intelligent decision making and handling of sensor data in order to be cost-effective but still reliable.

2.2 Comparative Study of Level-Sensing Technologies

2.2.1 Submersible Contact Probes vs. Non-Contact Ultrasonic Sensors

There are two main approaches for measuring water levels using electronic devices. One of them is based on using submersible contact probes that include metal conductors inserted into the water. As soon as the water connects the probes electrically, a circuit is completed. An obvious disadvantage of using contact probes is that because they are submerged in water, they are prone to corrosion and mineralization, thus failing after some period of time.

In order to overcome such a problem, the current project involves a non-contact ultrasonic transducer (HC-SR04). This transducer sends ultrasonic pulses into the water from above the tank surface. Knowing the time required for sound wave reflection, the device calculates the exact water level. There is no contact with the water; therefore, this kind of sensor does not corrode, providing its long life and additional security for its hardware.

2.2.2 Mechanical Float Switches vs. Electronic State Machines

In case of a mechanical float switch, only two states exist: ON and OFF. If the level of the water is low, then the pump starts; but in case the initial tank of water is exhausted, the float switch continues switching on the pump and causes heating and mechanical damage.

Currently, the mechanical float switch is replaced with an electronic state machine which is implemented in C++. The state machine works like a digital controller. Instead of simple detection of the presence of low water level, it checks simultaneously three different parameters: water level, the flow of water, and the time passed. If the computer program

realizes that there is no flow of water while the pump works, it turns off the pump to avoid any harm.

2.3 Requirements Analysis

Before embarking on writing codes and building any physical components, it is important to define what is expected of the system functionally and performance wise. The requirements will be categorized as Functional and Non-Functional requirements.

2.3.1 Functional Requirements (FR)

These are the core actions the system must perform to be considered successful:

- **FR1:** The system should open the supply inlet valve and turn on the water pump when the depth of water is 5.0 cm or below.
- **FR2:** The system should deactivate the water pump and shut off the inlet valve once the depth of water becomes full at 25.0 cm.
- **FR3:** The system should detect the flow of water. In case the water pump runs for 60 seconds and does not produce a flow rate of at least 0.1 Liters per minute, an emergency shutdown should occur to prevent pump dry running.
- **FR4:** The system must accept manual serial commands (such as STOP ALL or OVERRIDE ON) from a computer so a user can take manual control during maintenance.

2.3.2 Non-Functional Requirements (NFR) [Performance, Safety, Usability]

These define how well the system should perform its functions:

- **Performance:** The program must work without crashing; it should not freeze in order to ensure that the main execution cycle is non-blocking so that emergency messages may be received instantly.
- **Safety:** Data from the sensors should be processed via atomic reading of variables so that the microcontroller would not get access to corrupt data while counting pulses from the flow sensor.
- **Usability:** The OLED screen should display information in real time at a stable pace of 10 frames per second (FPS).

Chapter 2 - METHODOLOGY

Hardware Architecture & Electrical Design

3.1 System Hardware Overview

This project hardware architecture has been designed in order to link the digital logic working at low voltage levels with the high voltage actuator devices. This system is based on ESP32 microcontroller that is used for reading data from environmental sensors and then using that data to control 12V water pumps and solenoid valves via opto-isolated relays.

3.2 Component Selection & Justification

3.2.1 ESP32 Dual-Core Microcontroller (240 MHz, FreeRTOS Core)

The chosen CPU of the system was ESP32. Unlike 8-bit microcontroller like Arduino Uno, ESP32 features 32-bit dual-core microprocessor that works at a frequency of 240 MHz. Such a performance is required because the system should constantly receive data from hardware interrupts of the flow sensor and draw graphics on OLED screen simultaneously.

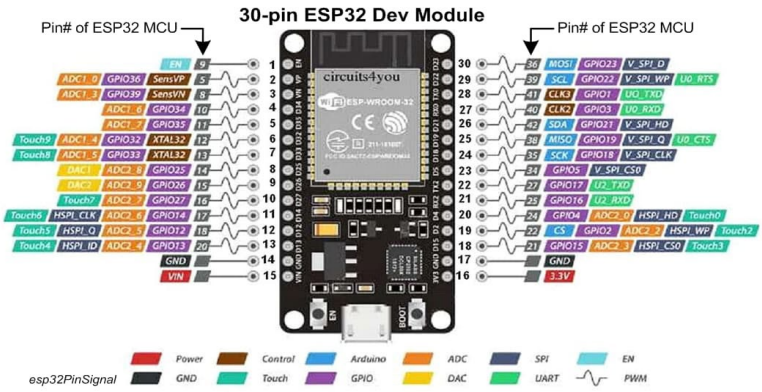


Figure 3.1: 30-pin ESP32 Microcontroller

3.2.2 ESP32 Expansion Board (Breakout Board)

There is the presence of mechanical vibrations in industrial or robotic environments, which easily makes loose jumper wires detach from standard breadboards used for prototyping. For the solution of this problem, an ESP32 Expansion Board is used as the hardware component of the circuit. An ESP32 microcontroller is mounted on this expansion board, and all GPIOs are separated in the form of headers with their own power and ground lines. This arrangement ensures that connections with each sensor and relay are not lost accidentally.

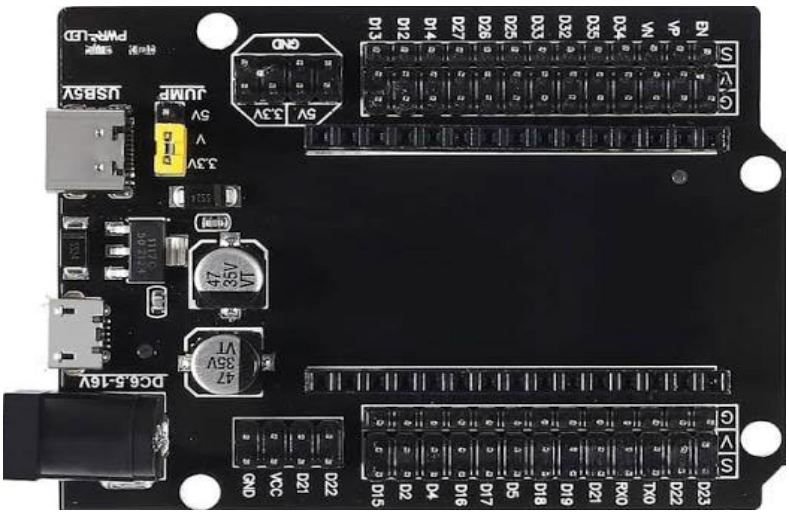


Figure 3.2: ESP32 Expansion Board

3.2.3 AJ-SR04T Waterproof Ultrasonic Sensor

The sensor makes use of ultrasonic waves in calculating the distance from the surface of the water. Unlike the typical sensors that are used indoors, the AJ-SR04T is provided with a totally sealed, waterproof probe, making it ideal for use in the tough, humid conditions inside a water tank. Before using it in the main project, the sensor was first subjected to a separate test on its accuracy in measuring distances. In the process, a notable hardware problem in the sensor was discovered; that is, the sensor is characterized by a large “blind spot” of 25.0 cm.

```
// 10µs acoustic trigger pulse
digitalWrite(TRIG_PIN, LOW); delayMicroseconds(2);
digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10);
digitalWrite(TRIG_PIN, LOW);

// Read echo with 30ms safety timeout and convert to cm
long duration = pulseIn(ECHO_PIN, HIGH, 30000);
float distance_cm = (duration * 0.0343) / 2.0;
```

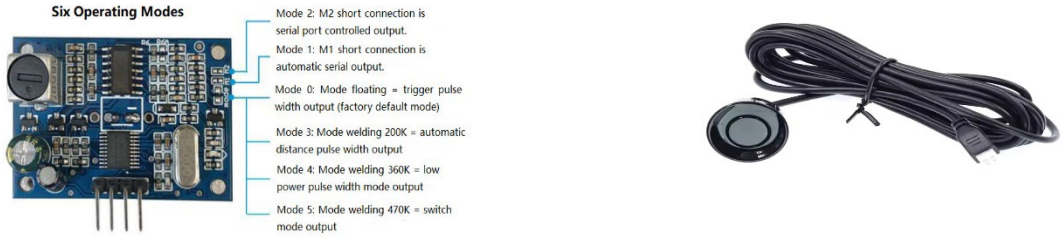


Figure 3.3: AJ-SR04T Waterproof Ultrasonic Sensor

3.2.4 YF-S201 Hall-Effect Flow Meter

The sensor is mounted on the water inflow pipe. There exists a small turbine within the sensor which reacts to the movement of the water and produces an electric signal which is then fed to the ESP32 for calculation of the accurate liters per minute (LPM).

```
// Define pin and volatile pulse counter for the interrupt
#define FLOW_PIN 15
volatile unsigned long pulseCount = 0;
// Interrupt Service Routine (ISR) triggered every time the turbine spins
void IRAM_ATTR countPulse() {
    pulseCount++;
}
// Attach the interrupt to look for the FALLING edge of the magnetic signal
attachInterrupt(digitalPinToInterrupt(FLOW_PIN), countPulse, FALLING);

// Formula: Flow rate in L/min = (Pulse frequency / 7.5)
float flowRate_LPM = (pulseCount / 7.5);
pulseCount = 0; // Reset the counter for the next second's reading
```



Figure 3.4: YF-S201 Water Flow Sensor

3.2.5 12V Solenoid Valves (IN/OUT) & Submersible DC Pump

TThere are two 12-volt solenoid valves that control the flow of water to the tank (IN), and the gravitational flow from the tank (OUT). The 12-volt submersible pump pumps the water through the IN valve.

```
// Define actuator pins
#define IN_VALVE_PIN 25
#define OUT_VALVE_PIN 26
#define PUMP_PIN 27

// --- Fill Sequence ---
digitalWrite(IN_VALVE_PIN, LOW);  // OPEN inlet valve
digitalWrite(PUMP_PIN, LOW);      // Turn PUMP ON
delay(3000);                      // Run for 3 seconds

// --- Stop & Drain Sequence ---
digitalWrite(PUMP_PIN, HIGH);     // Turn PUMP OFF
digitalWrite(IN_VALVE_PIN, HIGH); // CLOSE inlet valve
digitalWrite(OUT_VALVE_PIN, LOW); // OPEN gravity drain valve
```



Figure 3.5: 12V Solenoid Valve and Submersible DC Pump Actuator

3.2.6 Opto-Isolated 12V Relay Switching Module

As the ESP32 works at 3.3 volts, it can not power the 12-volt motors directly, and relays are used to interface between the motor and the microcontroller. During the testing of components, there was an observed boot-up problem where the relays were misfiring when the ESP32 was activated. This was sorted out using a hardware lock mechanism in the software:

```
// Define relay control pins
#define RELAY_1 25
#define RELAY_2 26

// This stops the sudden rush of power from triggering the 12V motors.
digitalWrite(RELAY_1, HIGH);
digitalWrite(RELAY_2, HIGH);

pinMode(RELAY_1, OUTPUT);
pinMode(RELAY_2, OUTPUT);
// --- Actuation Logic (Active-LOW) ---
digitalWrite(RELAY_1, LOW);  // Send 0V to turn Relay 1 ON
digitalWrite(RELAY_2, HIGH); // Send 3.3V to turn Relay 2 OFF
```

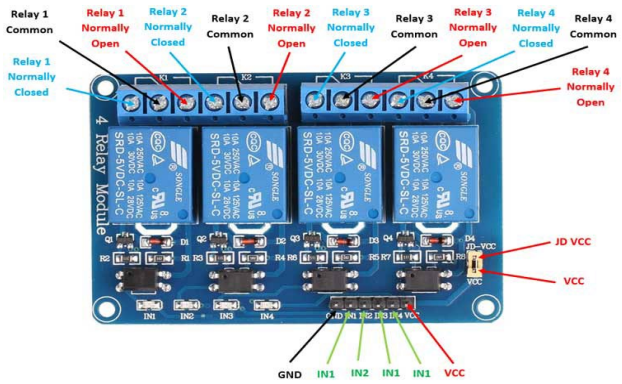


Figure 3.6: 4 Channel 12V Relay Module

3.2.7 0.96" SSD1306 I2C OLED Display

OLED Display acts as the dashboard of the system. For validating the I2C wiring (SDA and SCL) before making the main interface, a diagnostics code was used:

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// Create the display object (128x64 resolution)
Adafruit_SSD1306 display(128, 64, &Wire, -1);
// --- Safety Initialization ---
// Check if the screen is physically connected at the I2C address 0x3C
if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("OLED missing or wiring fault!");
    while (1); // Halt the entire system to prevent blind operation
}
// --- Screen Rendering Logic ---
display.clearDisplay();                  // Clear previous frames
display.setTextColor(SSD1306_WHITE);     // Set pixel color
display.setTextSize(1);                  // Set font size
display.setCursor(0, 10);                // Set X, Y coordinates
display.print("Flow: ");                 // Write string to buffer
display.print(flowRate_LPM);             // Write variable to buffer
display.display();                       // Push the buffer to the physical screen
```

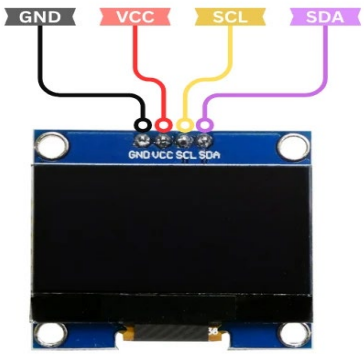


Figure 3.7: 0.96 Inches SSD1306 I2C OLED Display

3.2.8 Active Buzzer & LED Status Indicators

For providing the immediate physical awareness to the operator without the need to look at the screen, there is an active buzzer and a set of three LEDs (Red, Blue, Green). If the Green LED is on, then it means that the tank is full; Blue LED implies normal level. The Red LED along with the buzzer indicates critical low water level or fault in the system:

```
// Reset all visual indicators first
digitalWrite(LED_GREEN, LOW);
digitalWrite(LED_BLUE, LOW);
digitalWrite(LED_RED, LOW);

// Conditional Logic for visual and audio warnings
if (water_level <= 10.0) {
    digitalWrite(LED_RED, HIGH);        // Critical Low: Turn RED LED ON
    digitalWrite(BUZZER_PIN, LOW);      // Turn Buzzer ON (Active-LOW)
}
else if (water_level >= 46.0) {
    digitalWrite(LED_GREEN, HIGH);      // Tank Full: Turn GREEN LED ON
    digitalWrite(BUZZER_PIN, HIGH);     // Turn Buzzer OFF
}
else {
    digitalWrite(LED_BLUE, HIGH);       // Nominal Level: Turn BLUE LED ON
    digitalWrite(BUZZER_PIN, HIGH);     // Turn Buzzer OFF
}
```

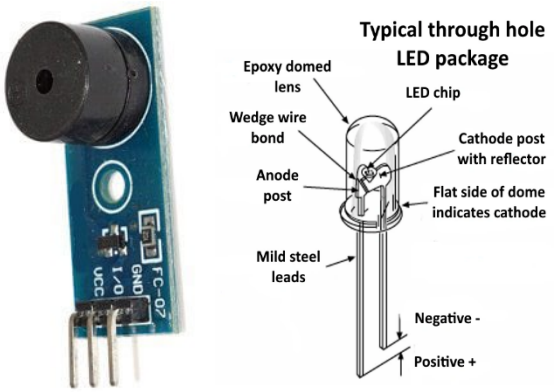


Figure 3.8: Active Buzzer Module and Normal Package of LED

3.2.9 LM2596 Buck Converter

The high-power devices like the 12V pump use a 12V source, while the relays and sensors work at 5V. The LM2596 buck regulator is a powerful power module that is responsible for reducing the voltage level from 12V to 5V. Thus, the power supply module will ensure the ESP32 and sensor circuits against voltage fluctuations due to the operation of the motor.

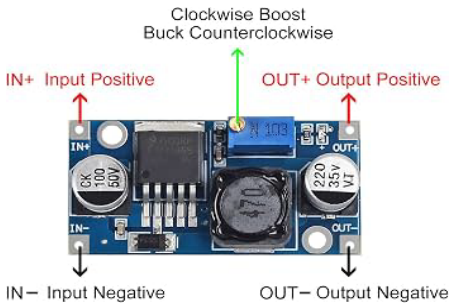


Figure 3.9: LM2596 Step-Down Voltage Buck Regulator

3.2.10 DC Female Barrel Power Jack

In order to connect the 12V high-power source from the external wall adapter to the power supply system, a DC female barrel jack can be used as the primary source of power. Thus, using this method ensures a secure connection that reduces the possibility of sparking and other electrical hazards caused by vibration of the pump.



Figure 3.10: Female DC Barrel Power Supply Jack

3.2.10 Jumper Wires /AWG 18 wires

For System connection and wiring part we used jumper wires and AWG 18 wires, we used jumper wires for circuit connections like esp23 to sensors, buzzer, led and relay and we used AWG 18 wires specially for high voltage power supply, like Solenoid valves, submersible DC Pumper and relay module.





Figure 3.11: Jumper wires / AWG 18 wires

3.3 Power Distribution Grid & Electrical Safety

The addition of the aqueous environment, 12 V actuator components, and 3.3 V microcontrollers requires thorough electrical insulation to avoid any short circuits.

3.3.1 Dual-Voltage Rail Architecture (12V Actuator Rail & 5V Logic Rail)

The power system design provides clear rail separation. In other words, the 12 V rail powers the pumps and valves regardless of the ESP32 electronics.

3.3.2 LM2596 Buck Converter Step-Down & Tuning

In order to provide power for large electromagnets of the relay module, 12 V voltage rail has to be converted to 5 V using an LM2596 buck regulator.

3.3.3 Relay Optocoupler Level Matching (3.3V Logic vs. 5V JD-VCC Coil)

Relay module has a jumper that separates the logic power from the motor power of the relay module. Output 5V of the buck converter is supplied to the JD-VCC pin to energize the relay coil while the ESP32's 3.3V pin is supplied to the VCC pin of the relay module.

3.3.4 Star Grounding Configuration & Common Ground Plane

All circuits require a common ground plane to function properly. The ESP32, relay, and buck converter are all connected via the buck converter's OUT- terminal as a common point for all ground planes in order to avoid any electrical interference.

3.3.5 Inductive Kickback Suppression (Flyback Diodes)

As the 12 V motor or solenoid is de-energized, the resulting collapsing magnetic field produces a voltage spike that may damage the microcontroller. Flyback diodes are used in parallel with the motor terminals in order to prevent this issue.

3.3.6 Ultrasonic Echo Voltage Divider Network (1 kΩ / 2 kΩ)

The ultrasonic HC-SR04 sensor has an operating voltage of 5V, while ESP32 GPIO pins can withstand only 3.3V. A simple voltage divider circuit using resistors of 1 kΩ and 2 kΩ converts 5V echo voltage to 3.3V, avoiding any damage to the ESP32.

3.4 Hardware Pinout Matrix & Schematic Interconnection

During initial testing, the pump and IN valve were assigned to GPIO pins 16 and 17, respectively. Later testing showed that these specific pins disrupt the PSRAM memory of the enhanced versions of ESP32, leading to system instability or crashes. In order to overcome this problem, the actuators were reassigned to other pins. Final pinout connections are listed below:

- **IN Valve Relay:** GPIO 26
- **Pump Relay:** GPIO 27
- **OUT Valve Relay:** GPIO 4
- **Flow Sensor:** GPIO 15
- **Ultrasonic TRIG / ECHO:** GPIO 5 / GPIO 18
- **Buzzer:** GPIO 13
- **LEDs (Green, Blue, Red):** GPIO 19, 23, 25
- **OLED Display:** GPIO 21 (SDA), GPIO 22 (SCL)[cite: 19]

UI/UX & SCADA Supervisory Control

4.1 High-Density Heads-Up Display (HUD) Design

When it comes to automation in industry, it becomes important for the operator to be able to know how the machine works without the necessity of connecting to the computer. In order to meet this need, a 0.96 inches OLED SSD1306 display was programmed to work as the Heads Up Display for the system. Since the main software loop makes use of non-blocking timers, the OLED provides 10 FPS of updates without any interference in collecting sensor data. The OLED display consists of three zones.

4.1.1 Dynamic Graphical Tank Percentage Bar

The left side of the screen contains a graphical bounding box, representing the physical water tank. The program calculates the present depth of the water and accordingly fills this bounding box with pixels in order to create an animated graphical bar which increases and decreases as per the changes in the depth of the water.

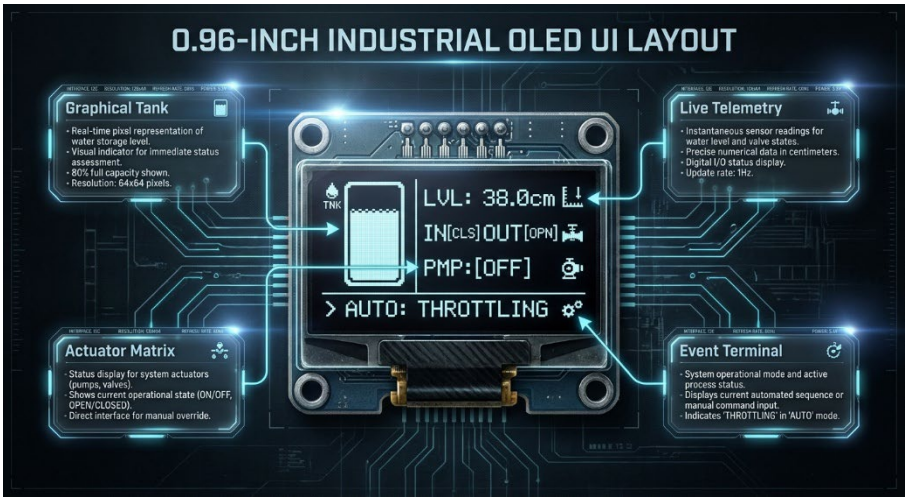


Figure 5.1: 0.96-inch Industrial OLED HUD Layout

Discussion

Reflection upon the project shows that in order to create this automation system, there had to be incorporation of digital software with hardware at high voltage, and this was not an easy task for four weeks.

- Hardware Interference: In the primary setup, the actuators were connected to GPIO 16 and 17, and due to PSRAM conflicts in the ESP32 chip, a system crash was experienced. This problem was sorted out by connecting the actuators to safe GPIOs (26, 27, and 4).
- Misfire of the Relays at Boot: The 12V relays would malfunction (open) briefly when the system was booted up. This problem was sorted out in code by setting the pins to HIGH first before configuring them as outputs.
- Limitations of the Sensor: The ultrasonic AJ-SR04T sensor has a physical limitation of a blind area of 25.0 cm, where any nearby objects cannot be sensed. Instead of acquiring another sensor, the program was mathematically adjusted such that the water level does not reach the blind area.
- Acoustic Noise: Ripples produced by the water inside the tank cause fluctuations in depth readings. This problem is resolved using a five-point moving average.

Achievement of Project Goals: The main goal of this project was the substitution of an unreliable mechanical float switch for a more sophisticated and automated fluid handling system. This goal was accomplished effectively..

- Stable Automation: Inclusion of an eight-state FSM makes the software error-free, making the system capable of automating the filling, throttling, and draining process independently.
- Safety: Prevention of hardware damage is ensured using software safety interlocks. One of the features of this system includes 60 seconds dry run burnout prevention in the pump, as well as 10.0 cm critical drain cut-off to prevent tank emptiness.
- Human-machine Interaction: The project is set to form an efficient human-machine interaction model. This includes the 0.96-inch OLED HUD and the SCADA serial interface for real-time data observation and control during emergencies.

Future Implementation

While the current Automated Water Tank system works perfectly as a localized unit, there are several ways it can be expanded in the future without needing to change the core ESP32 hardware.

For now, the SCADA interface requires a connection to the computer using the USB cable. Taking into account the fact that the ESP32 board has its own Wi-Fi, it would be possible to upgrade the system and create the possibility to connect it to a web server or to a cloud database, which will allow to remotely control the real-time OLED dashboard using any web browser.

Mobile application and push notifications: Instead of using just buzzer and LED indicators to inform about problems, it is possible to develop the mobile application. It will be possible to send the push notifications or SMS to the smartphone of a user in case of any serious problem, such as the lack of water in the pipe or dry-run of the pump.

Energy saving via machine learning: As the Liters Per Minute (LPM) is accurately counted thanks to the flow sensor, it would be possible to collect that information in future versions of software. Using the simple machine learning algorithms, the system will be able to identify the daily water consumption patterns and operate the pumps only during low electricity consumption periods.

Time Frame

Project Phase & Task Description	Start Date	End Date	Wk 1 (Jul 20–26)	Wk 2 (Jul 27–Aug 2)	Wk 3 (Aug 3–9)	Wk 4 (Aug 10–16)
PHASE 1: Research, Design & Unit Testing			■■■			
Literature Review & Requirements	Jul 20	Jul 21	■■■			
Hardware Acquisition	Jul 22	Jul 22	■■			
Cirkit Designer Schematics	Jul 23	Jul 23	■■			
Unit Test: OLED & I2C	Jul 24	Jul 24	■■			
Unit Test: Relay Boot Bug	Jul 25	Jul 25	■■			
Unit Test: PSRAM Pin Conflicts	Jul 26	Jul 26	■■			
PHASE 2: Software Architecture & Algorithms				■■■		
Non-Blocking millis() & UI Timers	Jul 27	Jul 28		■■■		
True Depth Math & Filter Algorithms	Jul 29	Jul 30		■■■		
Flow Interrupt Tuning (FALLING edge)	Jul 31	Jul 31		■■		
8-State FSM: enum Setup	Aug 1	Aug 1		■■		
8-State FSM: Transition Logic	Aug 2	Aug 2		■■		
PHASE 3: Integration, SCADA & Safety Interlocks					■■■	
Code Auto Fill Cycles	Aug 3	Aug 3			■■	
Code 60s Dry-Run Burnout Fault	Aug 4	Aug 4			■■	
Code Smart Throttling Limits	Aug 5	Aug 5			■■	
UI/UX: Graphical Tank	Aug 6	Aug 6			■■	
UI/UX: Telemetry & Actuator Matrix	Aug 7	Aug 7			■■	
SCADA: Serial Parser	Aug 8	Aug 8			■■	
SCADA: Emergency Overrides	Aug 9	Aug 9			■■	
PHASE 4: Calibration, Testing & Commissioning						■■■

AJ-SR04T Blind Spot Calibration	Aug 10	Aug 10	■
Volumetric Flow Bucket Test	Aug 11	Aug 11	■
Dry-Run Fault Simulation	Aug 12	Aug 12	■
Drain Cut-Off Simulation	Aug 13	Aug 13	■
10-Cycle System Endurance Run	Aug 14	Aug 14	■
Final Code Formatting (v12.0)	Aug 15	Aug 15	■
Project Report & Commissioning	Aug 16	Aug 16	■■

Figure 8.1: 4-Week Project Timeline

References

- Adafruit Industries, 2023. Monochrome OLED Breakouts and SSD1306. Online. Available from: https://learn.adafruit.com/monochrome-oled-breakouts. Accessed on 14 August 2026.

- Barr, M. and Massa, A., 2006. Programming Embedded Systems: With C and GNU Development Tools. 2nd ed. Sebastopol: O'Reilly Media. (Note: Reference is for Finite State Machine architecture and non-blocking timers.)

- Components101, 2021. YF-S201 Water Flow Sensor Datasheet and Pinout. Online. Available from: https://components101.com/sensors/yf-s201-water-flow-sensor. Accessed on 12 August 2026.

- espresso Systems, 2023. ESP32 Series Datasheet. Online. Available from: https://www.espressif.com/en/support/documents/technical-documents. Accessed on 10 August 2026.

- Handson Technology, 2020. 4-Channel 5V Optical Isolated Relay Module User Guide. Online. Available from: https://handsontec.com. Accessed on 13 August 2026.

- Smith, S. W., 1999. The Scientist and Engineer's Guide to Digital Signal Processing. San Diego: California Technical Publishing. (Note: Reference is for 5-point moving average filter algorithm.)

Appendix

Appendix A: Master OS v12.0 C++ Source Code

The following is the complete, non-blocking C++ source code for the ESP32 microcontroller, utilizing an 8-state FSM, hardware interrupts, and OLED I2C rendering.

Source link: [Click here]

Appendix B: Cirkit Designer Wiring Schematics

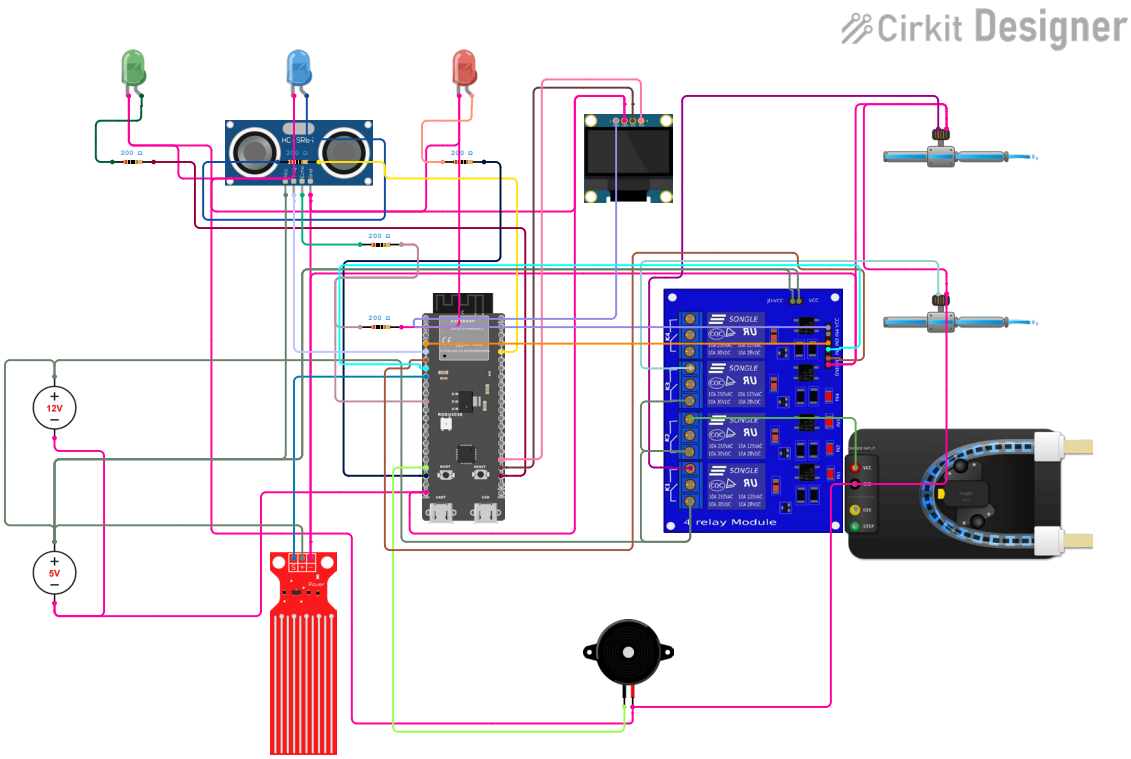


Figure 9.2: Cirkit Designer Wiring Schematics